Graph Exploration BFS

Contents

Saltar al contenido principal

14.1. Introducción

Al trabajar con grafos sin ponderar, la búsqueda en amplitud (BFS) es el algoritmo óptimo para encontrar el camino más corto entre los nodos. En este cuaderno, implementaremos BFS usando NumPy y NetworkX para encontrar el camino más corto en un grafo no ponderado y no dirigido.

14.2. ¿Cómo funciona BFS?

BFS explora un grafo nivel por nivel, partiendo de un nodo fuente:

1. Comenzamos en el nodo fuente y lo marcamos como visitado
2. Exploramos todos sus vecinos inmediatos
3. Luego pasamos al siguiente nivel de nodos (vecinos de vecinos)

Saltar al contenido principal

4. Este proceso continúa hasta que alcanzamos el nodo objetivo o visitemos todos los nodos accesibles

Dado que BFS explora los nodos en orden de distancia a la fuente, naturalmente encuentra el camino más corto en grafos sin ponderación.

14.3. Implementación con NumPy

Recuerda que está prohibido usar funciones o librerías integradas que implementen BFS. Necesitas implementar el algoritmo desde cero usando NumPy y listas estándar de Python. No son necesarias colas ni pilas para esta implementación, ya que podemos usar arrays NumPy para almacenar los nodos visitados y seguir el proceso de

Saltar al contenido principal

estructuras de datos de cola o pila será considerada inválida.

```
def bfs_numpy(graph, source, target)
    """
    Implementation of BFS using NumP

    Parameters:
    - graph: NetworkX graph (undirec
    - source: Source node
    - target: Target node

    Returns:
    - steps: Number of steps needed
    - shortest_path: List of nodes i
    """


    return steps, shortest_path
```

14.4. Ejemplo visual

Vamos a crear un grafo no dirigido sencillo

Saltar al contenido principal

```python
def create_example_graph():
    """Creates an example undirected
    G = nx.Graph()

    # Add nodes (cities)
    cities = ['Madrid', 'Barcelona',
    G.add_nodes_from(cities)

    # Add edges (no weights)
    edges = [
        ('Madrid', 'Barcelona'),
        ('Madrid', 'Valencia'),
        ('Madrid', 'Sevilla'),
        ('Madrid', 'Bilbao'),
        ('Madrid', 'Zaragoza'),
        ('Barcelona', 'Valencia'),
        ('Barcelona', 'Zaragoza'),
        ('Valencia', 'Sevilla'),
        ('Zaragoza', 'Bilbao')
    ]

    # Add the edges
    G.add_edges_from(edges)

    return G

# Create and visualize the graph
```

Saltar al contenido principal

```
# Visualize the graph
pos = nx.spring_layout(G, seed=42)
plt.figure(figsize=(10, 8))
nx.draw(G, pos, with_labels=True, nc
plt.title("Undirected Graph of Spani
plt.show()
```

14.5. Probando nuestro algoritmo BFS

Vamos a ejecutar nuestro algoritmo en este gráfico y ver los resultados:

```
# Define source and target nodes
source = 'Bilbao'
target = 'Sevilla'

# Run BFS
bfs_steps, bfs_visited = bfs_numpy(G
print(f"BFS Path from {source} to {t
print(f"Number of steps: {bfs_steps}
print(f"Visited nodes: {bfs_visited}
print(f"Number of nodes visited: {le
```

Saltar al contenido principal

```
print(f"\nNetworkX result:")
print(f"Shortest path: {path}")
print(f"Path length: {len(path) - 1}
```

14.6. Visualizing the BFS Process

Let's visualize the order in which BFS visits the nodes:

```
def visualize_shortest_path(G, short
    """Visualizes the shortest path
    plt.figure(figsize=(10, 8))

    # Create a copy of the graph for
    H = G.copy()
    pos = nx.spring_layout(H, seed=4

    # Create the edges of the shorte
    edge_colors = []
    edge_widths = []

    # Create edge labels dictionary
    edge_labels = {}
```

Saltar al contenido principal

```
    for u, v in H.edges():
        if len(shortest_path) > 1:
            # Check if this edge is
            is_path_edge = False
            for i in range(len(short
                if (u == shortest_pa
                    (v == shortest_pa
                    is_path_edge = T
                    # Add label with
                    edge_labels[(u,
                    break

            if is_path_edge:
                edge_colors.append(
                edge_widths.append(3
            else:
                edge_colors.append(
                edge_widths.append(1
        else:
            edge_colors.append('gray
            edge_widths.append(1.0)

    # Draw all nodes
    nx.draw_networkx_nodes(H, pos, n

    # Highlight the nodes in the sho
    nx.draw_networkx_nodes(H, pos, n
```

Saltar al contenido principal

```
        # Draw edge labels (numbers) for
        nx.draw_networkx_edge_labels(H,
                                     font

        # Draw node labels
        nx.draw_networkx_labels(H, pos)

        plt.title(title)
        plt.axis('off')
        plt.show()

# Visualize the shortest path with r
visualize_shortest_path(G, path, "Sh
```

14.7. Conclusion

In this notebook, we have explored the Breadth-First Search algorithm implemented using NumPy and NetworkX. BFS is a fundamental graph traversal method that systematically explores all vertices of a graph by visiting neighbors at each level before moving to the next level.

Saltar al contenido principal

to finding the shortest path in unweighted graphs, making BFS an optimal choice for such scenarios. Our implementation leverages NumPy's efficient array operations and NetworkX's graph representation capabilities, demonstrating how these tools can be combined to solve graph-related problems effectively. The visualization of the visit order provides an intuitive understanding of how BFS works and why it guarantees the shortest path in unweighted graphs. This algorithm forms the foundation for many more complex graph algorithms and is essential in network analysis, web crawling, social network analysis, and many other applications where finding the most direct connection between nodes is crucial.

Saltar al contenido principal

14.8. Exercise

14.8.1. Exercise 1

After we implemented the BFS algorithm using NumPy, now it’s time to understand the algorithm. What we want you to do is to explain the algorithm in your own words, and try to execute each step of the algorithm, showing print statements for each step. This will help you understand the algorithm better and see how it works in practice.

14.8.2. Exercise 2

Apply the BFS algorithm to the synthetic graphs that you generated in the previous notebook, and discuss the results. How does the algorithm perform on these graphs? Why it takes more steps to find the shortest path in some cases?

Saltar al contenido principal

13. Graph Generation

15. Graph Exploration DFS